

TEST SCRIPT GENERATOR

I have used VIBE CODING to change the existing extension to support PLAYWRIGHT + TYPESCRIPT and CYPRESS + TYPESCRIPT.

I Have added the prompts in the Green TextBox and the Final output from the extension in a Pink Text

As a QA Lead, understand the complete code. Go through the code one after another and explain me the logic behind this.

[MANDATORY] - Explain what theis code does. Where is the starting point. How the code traverse through different files.

[CRITICAL] Do not be more technical. Need only the Logic.

I'm tracing the app from the entry points first so I can explain the flow in the order the extension actually runs, then I'll walk file by file at a logic level.

I'm gathering the smallest set of files that define startup and navigation so the explanation follows the real execution path instead of the folder order.

I have the two entry points now: the extension manifest and the side panel page. Next I'm reading the scripts in the order they load so I can explain the handoff between them clearly.

The panel wiring is clearer now: one script handles saved settings and page connection, another only switches tabs. I'm reading the rest of the functional scripts next, especially the chat and inspector flow.

I've reached the core logic now: page inspection, prompt building, and AI generation. I'm reading the remainder of the chat flow plus the provider files so I can describe the full end-to-end path without going into implementation details.

What This Extension Does

This is a Chrome side-panel extension that helps a QA or automation user do one main job:

1. Pick elements from a live web page using an inspector.

2. Capture the selected page HTML.
3. Send that page content plus user instructions to an AI model.
4. Get back automation output such as: Feature file Page object Feature file plus Selenium step definitions

So in simple terms, this tool turns selected screen elements into test automation code.

Where It Starts

There are two real starting points.

1. Chrome starts from manifest.json This file tells Chrome:
 - the extension has a background worker
 - the side panel UI is panel.html
 - clicking the toolbar icon should use the extension
 - the extension is allowed to work on web pages
2. When the user clicks the extension icon, bg.js runs Its main job is simple:
 - open the side panel
 - listen for selected page content coming from the web page
 - store that selected content for later use
 - clean up highlights when the extension closes

So the execution path begins like this:

manifest.json → bg.js → panel.html

How The Code Moves Through the Files

Here is the full logic in the order the app actually works.

1. panel.html builds the whole side panel screen

This file creates:

- the Chat tab
- the Settings tab
- the Inspect button
- the Generate button
- provider and model dropdowns
- API key fields

It also loads the libraries and scripts that run the extension logic.

2. Theme and app identity are applied first

These files are only about app-wide colors and branding:

- `src/config/appConfig.js`
- `src/config/configUtils.js`
- `src/scripts/init-colors.js`
- `src/scripts/init-config.js`

Their logic is:

- keep one central color setup
- apply those colors as soon as the panel opens
- avoid the panel flashing with wrong colors at startup

This is visual setup only, not business logic.

3. `src/scripts/popup.js` prepares the panel to talk to the current browser tab

This file is the panel initializer. Its logic is:

- find the active browser tab
- inject the page helper script into that tab if needed
- create or restore the connection with that page
- reload saved settings like provider, model, and API keys
- show the correct API key field based on provider
- reconnect if the page or tab changes
- send cleanup messages when the panel closes

So this file acts like the bridge between the side panel and the live web page.

4. `src/scripts/tabs.js` only switches between Chat and Settings

This file is very simple:

- when the user clicks a tab button, it shows that tab and hides the other one

No generation logic happens here.

5. `src/content/content.js` is the page inspector that runs inside the actual website

This is one of the most important files.

Its logic is:

- wait until the extension asks to start inspection
- when inspection is on, highlight page elements as the mouse moves
- when the user clicks an element, mark it as selected
- allow multiple elements to be selected
- collect the HTML of all selected elements
- send that combined HTML back to the extension
- remove highlights and clean up when asked

So this file is how the extension “reads” the page that the user wants code for.

Flow here is:

Chat panel asks to inspect → `src/content/content.js` lets the user select page elements → selected HTML is sent back

6. `bg.js` receives the selected page content

When the page helper sends selected content:

- background receives it
- stores it
- keeps it available for the rest of the extension

This means the selected page content survives long enough to be used during generation.

7. `src/scripts/chat.js` is the main brain of the side panel

This is the core user flow file.

Its logic is:

- manage chat messages on the screen
- manage the Inspect button state
- manage the Generate button
- load saved provider, model, and API keys

- receive the selected page content
- decide what type of output to generate
- send requests to the chosen AI provider
- display the response nicely
- allow reset and copy actions

In business terms, this file controls the whole user journey: Inspect → capture page content → combine with instructions → call AI → show generated result

8. `src/scripts/prompts.js` decides what the AI should generate

This file contains the instruction templates.

Current logic supports mainly these outputs:

- Selenium Java Page Object only
- Cucumber Feature file only
- Cucumber Feature plus Selenium Java step definitions

When Generate is clicked, `src/scripts/chat.js` checks:

- selected language
- selected browser engine
- whether Feature is checked
- whether Page Class is checked

Then it picks the matching prompt from `src/scripts/prompts.js`.

Important QA point: The current real support is mainly Java plus Selenium. Other combinations appear in the UI, but the logic mostly warns that they are not fully supported yet.

9. `src/scripts/chat.js` prepares the final request

Before sending to AI, it does this:

- takes the selected page HTML
- shortens large dropdowns so the input is not too big
- gets the current page URL
- adds the user's extra instructions from the text box

- builds one or more prompts depending on the selected mode

Then it sends those prompts to the chosen provider.

10. Provider files only handle the AI call

These files are simple connectors:

- `src/scripts/api/groq-api.js`
- `src/scripts/api/openai-api.js`
- `src/scripts/api/testleaf-api.js`

Their logic is:

- accept the final prompt
- send it to the provider
- get the response back
- return the generated content to chat

They do not decide what to generate. They only send and receive.

11. `src/scripts/chat.js` shows the final output

Once AI responds:

- the loading state is removed
- generated output is shown in the chat window
- token usage is shown
- copy action is made available
- selected page content is cleared
- inspector state is reset

That completes one generation cycle.

Simple End-To-End Journey

If I explain the whole app in one plain sequence, it is this:

1. Chrome reads `manifest.json`.
2. User clicks the extension icon.
3. `bg.js` opens `panel.html`.

4. panel.html loads the UI and scripts.
5. src/scripts/popup.js connects the panel to the active page.
6. User clicks Inspect.
7. src/content/content.js lets the user select elements on the website.
8. Selected HTML is sent back through the extension.
9. src/scripts/chat.js stores that selected content.
10. User clicks Generate.
11. src/scripts/chat.js asks src/scripts/prompts.js for the right instruction template.
12. src/scripts/chat.js sends the request through one provider file.
13. The AI response comes back.
14. src/scripts/chat.js displays the generated code in the panel.

What Each Folder Is For

- assets: logos and images only
- lib: third-party display helpers These are used to format and color the generated code in the panel
- src/config: app settings for look and branding
- src/content: logic that runs inside the actual website
- src/scripts: side panel behavior and AI flow
- src/styles: styling only, no business logic

QA-Level Summary

From a QA Lead view, the main business logic is:

- open side panel
- connect to active page
- let user inspect and select page elements
- capture selected HTML
- choose generation type
- send prompt to selected AI provider
- display generated automation output

The most important functional files are:

- manifest.json
- bg.js
- panel.html

- src/scripts/popup.js
- src/content/content.js
- src/scripts/chat.js
- src/scripts/prompts.js
- src/scripts/api/groq-api.js
- src/scripts/api/openai-api.js
- src/scripts/api/testleaf-api.js

If you want, I can next give you:

1. a very simple file-by-file explanation in the exact loading order, or
2. a QA test flow showing what scenarios should be tested from launch to code generation.

As a QA Lead access the file impact, when we want to add the Playwright + typescript and cypress with typescript combination.

I'll map exactly which files control language-engine combinations and prompt selection, then give you a QA-focused impact list for adding Playwright+TypeScript and Cypress+TypeScript. I'm quickly collecting line-level references so you can estimate scope and test impact confidently.

Searched for regex

getPromptKeys|isJavaSelenium|unsupported|languageBinding|browserEngine|javaGenerationMode (**/src/scripts/chat.js), 17 results

Searched for regex

SELENIUM_JAVA_PAGE_ONLY|CUCUMBER_ONLY|CUCUMBER_WITH_SELENIUM_JAVA_STEPS|DEFAULT_PROMPTS (**/src/scripts/prompts.js), 8 results

Searched for regex languageBinding|browserEngine|providerSelect|modelSelect (**/panel.html), 8 results

Searched for regex

modelsByProvider|updateModelOptions|selectedProvider|selectedModel (**/src/scripts/popup.js), 17 results

For adding Playwright + TypeScript and Cypress + TypeScript, the impact is mostly in the decision logic and prompt library, not in extension startup.

Primary File Impact (Must Change)

1. chat.js
2. This is the core decision point that currently routes generation mainly to Java + Selenium flow.
3. Impact:
 - Add new routing logic for:
 - TypeScript + Playwright
 - TypeScript + Cypress
 - Remove/adjust current “unsupported combination” path for these two combos at chat.js.
2. prompts.js
This is the source of generation templates.
Impact:
 - Add new prompt templates for:
 - Playwright TypeScript page/class output
 - Cypress TypeScript page/class or spec output
 - Feature + step-definition style variants if needed
 - Add corresponding keys in the type map at prompts.js.

Secondary Impact (Likely Change)

1. panel.html and panel.html

UI already has TypeScript, Playwright, Cypress in dropdowns.

Impact:

- Usually no structural UI change needed.
- Only change if you want clearer labels like “Supported combinations only”.

2. popup.js

This controls provider/model dropdown and saved settings.

Impact:

- No direct combo logic here.
- Change only if model recommendations should depend on Playwright/Cypress selection.

Low or No Impact

1. manifest.json

No expected impact for this feature.

2. bg.js

No expected impact; it only handles side panel open, messages, and storage relay.

3. content.js

No expected impact; element capture works the same regardless of target framework.

4. Provider connectors:

- openai-api.js
- groq-api.js
- testleaf-api.js

No functional impact unless response format rules change.

Logic Flow Impact Summary

1. Current flow already captures DOM and user input correctly.
2. The gap is only in “what prompt key gets selected for a language+engine pair”.
3. Once new prompt keys are added and mapped, the same pipeline will generate Playwright/Cypress TypeScript output.

QA Impact Areas (What To Validate)

1. Combination routing:
 - TypeScript + Playwright picks only Playwright prompts.
 - TypeScript + Cypress picks only Cypress prompts.
 - Existing Java + Selenium behavior stays unchanged.
2. Output correctness:
 - Generated code format is TypeScript syntax.
 - Framework-specific style is correct (Playwright vs Cypress).
3. Negative coverage:
 - Unsupported combos still show warning properly where intended.
 - No fallback to wrong prompt key.
4. Regression:
 - Inspector selection, DOM capture, provider/model save/load, and reset flow remain stable.

If you want, I can give a compact QA test matrix (happy path + edge cases) specifically for these two new combinations.

List the file which has impact for this change

Primary impact files for adding Playwright + TypeScript and Cypress + TypeScript:

- chat.js
- prompts.js

Potential/conditional impact files (only if you want UX or model-selection behavior changes):

- panel.html
- popup.js

As a QA Lead, Could you please list what has to be changed in these files to incorporate the changes to include Playwright + Typescript and Cypress + Typescript

Must Change

1. chat.js
2. What to change:
 - Expand the combination decision logic so these are treated as supported:
 - Playwright + TypeScript
 - Cypress + TypeScript
 - Add new checks similar to existing Java/Selenium check, then route each combo to the correct new prompt keys.
 - Update the unsupported message path at chat.js so it no longer triggers for these two combos.
 - Keep existing Java/Selenium path unchanged to avoid regression.

2. prompts.js

What to change:

- Add new prompt templates for Playwright + TypeScript output.
- Add new prompt templates for Cypress + TypeScript output.
- Add prompt keys in the generator type map at prompts.js so chat routing can call them.
- Ensure prompt intent matches your two generation modes in UI:
 - Feature file
 - Page/Class code (or equivalent framework output)

Change If You Want Better UX (Optional but Recommended)

1. panel.html and panel.html

What to change:

- Keep current dropdown options (already present), but update helper text/labels if needed to make supported combinations explicit.
- If desired, rename the current mode labels so they are framework-neutral (today they are Java-oriented).

2. popup.js

What to change:

- No core combo logic change required.
- Optional: add guardrails so provider/model selection remains valid when user switches to Playwright/Cypress TypeScript flows.
- Optional: preserve/reset model choices more clearly per framework if your QA strategy requires that behavior.

QA Validation After Change

1. TypeScript + Playwright selects only Playwright prompts.
2. TypeScript + Cypress selects only Cypress prompts.
3. Java + Selenium still works exactly as before.
4. Unsupported warning appears only for truly unsupported combinations.

Make changes in prompt.js now, to add for

PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY

PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER

CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS

Same do for Cypress and typescript. Taka JAVA + SELENIUM as example and make the changes.

I'm applying the changes directly in prompts.js now by adding the new Playwright/Cypress TypeScript prompt templates and registering their generator keys, using the existing Java+Selenium format as the pattern. After the edit, I'll quickly validate the file for syntax issues.

Generating patch (292 lines) in

The prompt templates are now added in prompts.js, including all three Playwright+TypeScript keys and the same three for Cypress+TypeScript, plus their generator-type registrations. I'm running a quick error check on this file to ensure no syntax issues were introduced.

Checked , no problems found

Updated successfully in prompts.js.

Added prompt templates for Playwright + TypeScript:

- PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
- PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER
- CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS

Added prompt templates for Cypress + TypeScript:

- CYPRESS_TYPESCRIPT_PAGE_ONLY
- CYPRESS_TYPESCRIPT_WITH_CUCUMBER
- CUCUMBER_WITH_CYPRESS_TYPESCRIPT_STEPS

Also updated CODE_GENERATOR_TYPES with all six new keys in prompts.js.

Validation:

- Syntax check completed for prompts.js
- No errors found

Next required step to make these usable at runtime:

1. Update prompt-key routing in chat.js so TypeScript+Playwright and TypeScript+Cypress combinations map to these new keys.

Made changes.

As a QA Lead do the following in the chat.js file

Expand the combination decision logic so these are treated as supported:

Playwright + TypeScript

Cypress + TypeScript

Add new checks similar to existing Java/Selenium check, then route each combo to the correct new prompt keys.

Update the unsupported message path at chat.js:617 so it no longer triggers for these two combos.

Keep existing Java/Selenium path unchanged to avoid regression.

I'm updating chat.js now to support the two new TypeScript combinations while preserving the existing Java+Selenium path exactly as-is. I'll first patch only the prompt-routing logic and helper checks, then run an error validation.

Read , lines 520 to 700

I've isolated the routing block. Next I'm applying a targeted patch that keeps Java+Selenium untouched, adds helper checks for TypeScript+Playwright/Cypress, and maps each mode to the new prompt keys so unsupported warnings won't appear for these combos.

Generating patch (70 lines) in

Checked , no problems found

Updated in chat.js as requested.

What I changed:

1. Expanded combination support logic in chat.js
 - Added support for:
 - Playwright + TypeScript
 - Cypress + TypeScript
2. Added helper checks (same pattern as existing Java/Selenium)
 - Added:
 - isTypeScriptPlaywright(...)
 - isTypeScriptCypress(...)
3. Routed each mode to the correct new prompt keys
 - No mode selected fallback:
 - Playwright+TS -> PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
 - Cypress+TS -> CYPRESS_TYPESCRIPT_PAGE_ONLY
 - Feature + Page selected:
 - Playwright+TS -> CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS
 - Cypress+TS -> CUCUMBER_WITH_CYPRESS_TYPESCRIPT_STEPS
 - Feature only:
 - Playwright+TS -> PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER
 - Cypress+TS -> CYPRESS_TYPESCRIPT_WITH_CUCUMBER
 - Page only:
 - Playwright+TS -> PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
 - Cypress+TS -> CYPRESS_TYPESCRIPT_PAGE_ONLY
4. Unsupported warning behavior updated implicitly
 - The existing unsupported path now skips these two combinations because they are recognized as supported.
 - Java + Selenium path remains unchanged to avoid regression.

Validation:

- Checked chat.js for errors.
- No errors found.

Made changes.

make below changes to to panel.html

What to change:

Keep current dropdown options (already present), but update helper text/labels if needed to make supported combinations explicit.

If desired, rename the current mode labels so they are framework-neutral (today they are Java-oriented).

I'm updating panel.html to make supported combinations explicit and to neutralize Java-specific mode wording, while keeping existing dropdown options unchanged. I'll make only label/text adjustments so behavior stays intact.

Read

I found the right spots: the generation mode checkbox labels and settings labels. I'm applying a minimal text-only update to make supported combinations explicit and framework-neutral without touching IDs or behavior.

Generating patch (56 lines) in

Checked , no problems found

Updated panel.html with text/label-only changes as requested.

What changed:

1. Framework-neutral generation mode labels
 - “Feature File” -> “BDD Feature File”
 - “Page Class” -> “Page Object / Page Class”
 - Added a short helper line under these options to clarify usage.
2. Settings label cleanup
 - “Supported Language” -> “Language”
3. Explicit supported-combination guidance

- Added helper text under Browser Engine:
 - Java + Selenium
 - TypeScript + Playwright
 - TypeScript + Cypress

What stayed unchanged:

- All dropdown options remain exactly as they were.
- All IDs/names used by scripts are untouched, so behavior remains stable.

Validation:

- panel.html checked.
- No errors found.

Made changes.

make changes in popup.js if needed

I'm checking popup.js specifically for language-engine settings handling to confirm whether any guardrails are missing after the new TypeScript combinations. If needed, I'll add only minimal compatibility logic without changing existing provider behavior.

Searched for regex

languageBinding|browserEngine|selectedLanguage|selectedEngine|storage.get|storage.set (**/src/scripts/popup.js), 7 results

I found a gap: popup.js currently does not manage language-engine compatibility at all. I'm adding a small settings guard so unsupported pairs auto-correct to a supported combination and persist, which aligns with the new support matrix.

Read , lines 1 to 320

I'm making a lightweight, safe update in popup.js: persist and restore Language + Browser Engine selections so the new TypeScript combinations remain sticky across panel sessions. This avoids behavior surprises without altering existing provider/model logic.

Generating patch (58 lines) in

Checked , no problems found

Made a needed, low-risk update in popup.js.

What I changed:

1. Added settings element references
 - languageBinding
 - browserEngine
2. Persisted selection changes to storage
 - selectedLanguageBinding
 - selectedBrowserEngine
3. Restored saved selections on load
 - Reads both keys from storage during initialization
 - Re-applies them to the dropdowns if present

Why this is needed:

- Ensures Playwright + TypeScript and Cypress + TypeScript selections stay consistent across panel reopen/reload.
- Keeps behavior predictable for QA validation without changing existing provider/model flows.

Validation:

- Checked popup.js
- No errors found

If you want, next I can add strict combo guardrails in popup (auto-correct unsupported pairs) so users can only pick supported combinations.

OUTPUT FILE

Feature: Login functionality

Scenario Outline: Successful login with valid credentials

Given I open the Kyro application login page

When I fill in the username "<username>" and password "<password>"

And I click the "Login with Password" button

Then I should be redirected to the home page

Examples:

| username | password |

| ramkumar | S3cure!2023 |

| snehal | Pass@2023! |

| vijaykumar | MyPass#2023 |

| lakshmi | Lakshmi@123 |

| priyanka | Priy@2023! |

```
import { Given, When, Then, And } from '@badeball/cypress-cucumber-preprocessor';
```

```
import '@cypress/xpath';
```

```
// Open the login page
```

```
Given('I open the Kyro application login page', () => {
```

```
  cy.visit('https://stage.kyro.ai/login');
```

```
});
```